



Chapter 2

Logic Algebra





Outline:

- ☆ Basic concepts;
- ☆ Basic properties and rules;
- ☆ Representations;
- ☆ Reduction of Boolean algebra





In 1847, The British mathematician G. Boole proposed the use of mathematical analysis to represent the logical structure of the propositional statement, and converted the formal logic to an algebraic calculus, thus creating the famous "Boolean algebra".

In 1938, C. E. Shannon applied Boolean algebra to the switch circuit of telephone relay, thus introduced the "switch algebra".

With the development of electronic technology, integrated circuit logic gates have replaced mechanical contact switches, so people usually called switch algebra as logic algebra.

Logic algebra is the theoretical basis and important mathematical tool for logical design of digital systems.





2.1 Logic functions

Logic algebra L is a mathematical system that defines a series of logical operations (*AND*, *OR*, *NOT*), performed on sets of variables K , constants (0 and 1). It is represented as $L = \{K, +, \cdot, \overline{}, 0, 1\}$.

I, Binary variable

A binary variable is like a variable in regular algebra, can be designated by letters, symbols or their combination; However, there are two differences:

1. The variable can only have the values: **0 and 1**;
2. Logic values, 0 and 1, are formal symbols used to determine whether an event is true or false; rather than to present **magnitudes or positive/negative**.





II, Primitive logic functions

To describe a digital system, we must represent the relationships amongst the switch elements in the system; the relationship can be described as a number of logic functions in mathematics.

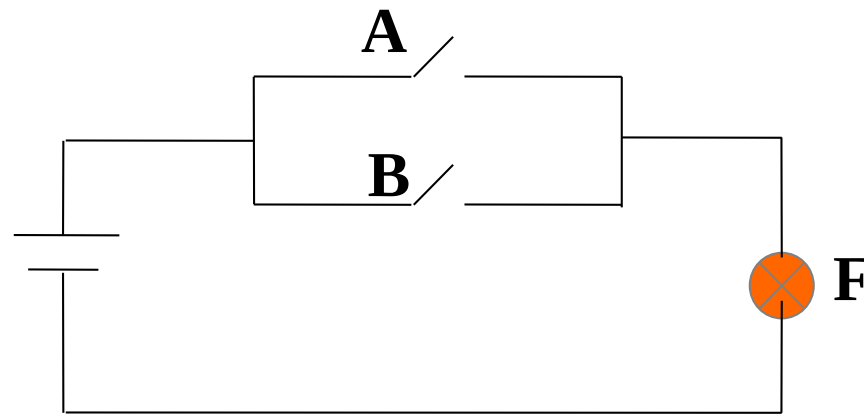
Three logic functions (AND, OR, NOT or complement) provide the foundation for all digital systems' analysis and design.

1. “OR” function

If multiple conditions can determine whether an event occurs, as long as one or more conditions are true, the event will occur. Such a causal relationship is called “OR” logic.

E.g., a circuit using two switches to parallelly control a light:





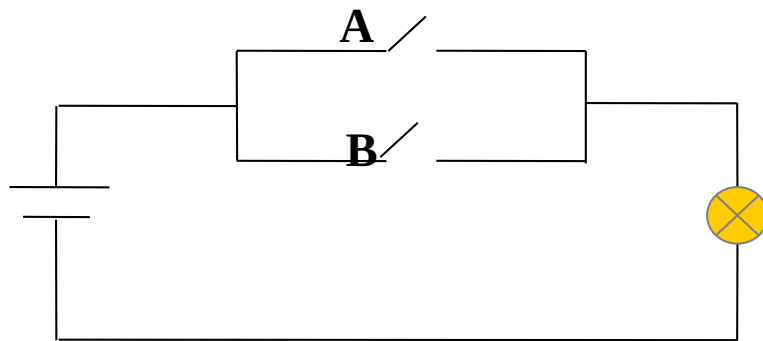
In the circuit, Switches A and B control light F in parallel. When one of the Switches A and B is on or both are on, the light F is on. Therefore, the relationship between the light F and the switches A and B is an OR logic, which can be expressed as

$F = A + B$ is read as **F equals A OR B**





Assume that the switch is off with 0, and on with 1; the light is off with 0, and on with 1, then the relationship between light F and Switches A and B is shown in the following table. i.e., : **If only one of A and B is 1, then F is 1; only when A and B are 0, F is 0.**



Parallel circuit

“OR” Truth Table

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

“OR” function’s arithmetic rules:

$$0 + 0 = 0 \quad 1 + 0 = 1$$

$$0 + 1 = 1 \quad 1 + 1 = 1$$



A logical circuit realizing the OR operation relationship is called an OR gate.





2. “AND” function

If multiple conditions must occur at the same time for an event to occur, the relationship is called "AND" logic.

Two input AND function are expressed as

$$F = A \cdot B \quad \text{or} \quad F = AB$$

i.e.: if A and B are 1, then F is 1; otherwise F is 0.

“AND” Truth table

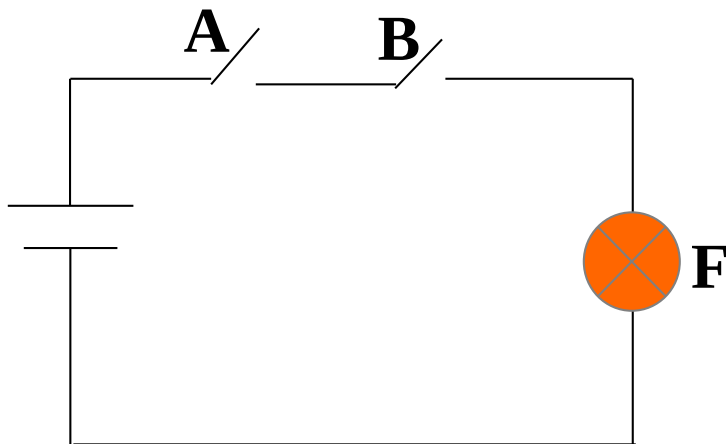
A	B	F
0	0	0
0	1	0
1	0	0
1	1	1





Two switches control the light in series. Obviously, the light can only be on when both switches are closed, otherwise the light is off.

Assume that the switch is off with 0, and on with 1; the light is off with 0, and on with 1, then the relationship between light F and Switches A and B is shown in the following table. i.e., A and B control a the light F, the lights can only be on when both switches are closed, otherwise the lights is off.



Series circuit

“AND” function’s arithmetic rules:

$$\begin{array}{ll} 0 \cdot 0 = 0 & 1 \cdot 0 = 0 \\ 0 \cdot 1 = 0 & 1 \cdot 1 = 1 \end{array}$$

In the digital system, an AND gate is a circuit that performs the AND logic function.





3. “NOT” function

If the occurrence of an event depends on the negation of its condition, such a relationship is called “Not” logic.

In logic algebra, the “NOT” function symbol is “ $\neg$ ”, the function is expressed by:

$$F = \overline{A}$$

The expression is read as F equals A NOT

i.e., if A is false (0), then, F is true (1);

if A is true (1), then F is false (0)

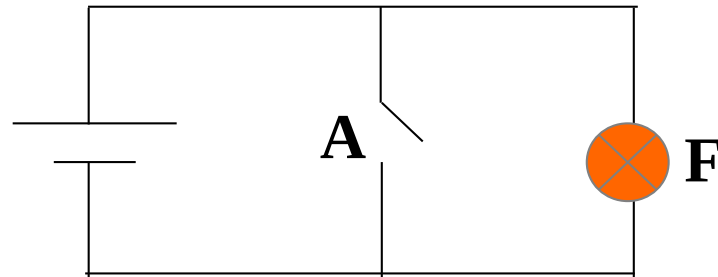
“Not” function’s truth table

A	F
0	1
1	0





e.g., we have a circuit, in which the switch and light are connected in parallel. The light is on only when the switch is off. If the switch is on, the light is off. **Assume that the switch is off with 0, and on with 1; the light is off with 0, and on with 1, thus, the circuit's relation is represented as a NOT function.**



Parallel circuit of a switch and a light

“NOT” function’s arithmetic rules:

$$\bar{0} = 1 \quad ; \quad \bar{1} = 0$$

A logic circuit that implements “NOT” function in digital systems are called “NOT” gate, sometimes called “Inverter.”





2.2 Basic properties

Logic algebra L is a mathematical system that defines a series of logical operations(AND, OR, NOT), performed on sets of variables (0 and 1), K constants. Thus, $L=\{K,+, \cdot, -, 0, 1\}$. **The system has the following properties:**

Property 1

For any logic variables, A 、 B , we have

$$A + B = B + A \quad ; \quad A \cdot B = B \cdot A$$

Property 2 Associative

For any logic variables, A 、 B and C , we have

$$(A + B) + C = A + (B + C)$$

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$





Property 3 Distributive

For any logic variables, A、B, we have

$$A + (B \cdot C) = (A + B) \cdot (A + C) ;$$

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

Property 4 Constants

for any variable, we have

$$A + 0 = A ; A \cdot 1 = A$$

$$A + 1 = 1 ; A \cdot 0 = 0$$

Property 5 Complement

For any variable A, a unique $\bar{A}$ exists

$$\bar{A} + A = 1 \quad \bar{A} \cdot A = 0$$





2.1.2 Logic functions & Equivalence between logic functions

I. Definition of logic function

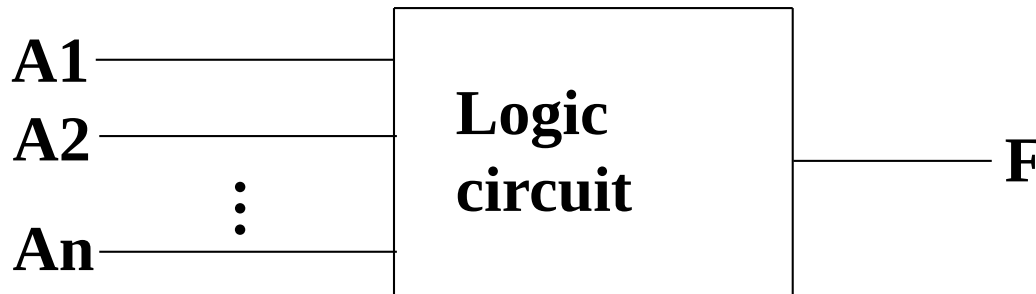
The definition of a function in logic algebra is similar to the definition of a function in the ordinary algebra, that is, the function's result changes with the input variables. But compared with the concept of functions in ordinary algebra, the logic function has the following characteristics:

- 1. Values of both constants and variables must be 0 or 1 ;**
- 2. The relationship between functions and variables is determined by the three basic operations of "OR", "AND", and "NOT".**





Assume a logic circuit's input are $A_1, A_2, \dots, A_n$, output logic variable is F :



A generalized logic circuit

where, F is called the function of $A_1, A_2, \dots, A_n$, F is defined as:

$$F = f(A_1, A_2, \dots, A_n)$$

The output function of the logic circuit is determined by the values of logical variables and the structure of the logic circuit itself.





Just like functions in the ordinary algebra, the logic functions also contains the problem that whether two functions are equal or not. Assume we have two functions:

$$F_1 = f_1(A_1, A_2, \dots, A_n)$$

$$F_2 = f_2(A_1, A_2, \dots, A_n)$$

For any set of logic variables $A_1, A_2, \dots, A_n$, the value of F_1 is equal to that of F_2 , then we can say function F_1 is equal to F_2 , it is expressed as $F_1 = F_2$.

How to determine if two logic functions are equal?

Two methods: **Truth table method;**
Algebraic method





2.1.3 Representation of logic function

How to describe a logic function?

The common methods are: logic expression, **truth table**, **Karnaugh map**.

I. Logic expression

A logic expression consists of logic variables and the three basic operation symbols (OR, AND, NOT). E.g.,

$$F = f(A, B) = \bar{A}B + A\bar{B}$$

Relationships between F and variables A and B are:

When the values of variables A and B are different, the value of function F is "1"; when the values are the same, the value of function F is "0".





Shorthand for logic expressions

1. No parenthesis under the “NOT” e.g. $\overline{A \cdot B}$, $\overline{A + B}$

2. “AND” can be omitted, e.g., $A \cdot B$ can be written as AB

3. In a expression containing both “AND” and “OR”, we need calculate “AND” first, then “OR”, the parentheses can be omitted. E.g.,

$(A \cdot B) + (C \cdot D)$ can be written as $AB + CD$.

Note that: $(A+B) \cdot (C+D)$ cannot be written as $A+B \cdot C+D$!

precedence: $() \longrightarrow - \longrightarrow \cdot \longrightarrow \oplus \longrightarrow +$
High $\xrightarrow{\hspace{10em}}$ low

4. $(A+B)+C$ or $A+(B+C)$ can be replace by $A+B+C$; $(AB)C$ or

$A(BC)$ can be replaced by ABC .





II. Truth table

A table listing the combination of the values of all input variables of a logic function and their corresponding function values in turn is called a truth table.

A logic function with n variables, its truth table contains 2^n row. E.g.:

Truth table of $F = A\bar{B} + \bar{A}C$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

A truth table consists of two components:

The left column lists all the possible values of the variables. In order to avoid omission, the values of the variables are usually given in binary digital order; the right column is the logical function value.





III Karnaugh map

The Karnaugh map is a matrix of squares, each square denotes a possible combination of all variables.

This graphic representation of describing logic functions is very useful in simplifying logic functions. It will be introduced in detail later with the problem of function simplification.

Three representation methods for describing logic functions can be used for different occasions. But for a specific problem, they are just different descriptions of the same thing, and they can be easily transformed with each other.





2.2 Theorems and rules of Boolean algebra

2.2.1 Theorems

Eight common used theorems:

Theorem 1

$$0 + 0 = 0 \quad 1 + 0 = 1 \quad 0 \cdot 0 = 0 \quad 1 \cdot 0 = 0$$

$$0 + 1 = 1 \quad 1 + 1 = 1 \quad 0 \cdot 1 = 0 \quad 1 \cdot 1 = 1$$

$$\bar{1} = 0$$

$$\bar{0} = 1$$





Theorem 2 $A + A = A$; $A \cdot A = A$

Prove: $A + A = (A + A) \cdot 1$ Property 4

$= (A + A) \cdot (A + \bar{A})$ Property 5

$= A + (A \cdot \bar{A})$ Property 3

$= A + 0$ Property 5

$= A$ Property 4

Theorem 3 $A + A \cdot B = A$; $A \cdot (A + B) = A$

Prove: $A + A \cdot B = A \cdot 1 + A \cdot B$ Property 4

$= A \cdot (1 + B)$ Property 3

$= A \cdot (B + 1)$ Property 1

$= A \cdot 1$ Property 4

$= A$ Property 4





4 $A + \bar{A} \cdot B = A + B ; \quad A \cdot (\bar{A} + B) = A \cdot B$

Prove $A + \bar{A} \cdot B = (A + \bar{A}) \cdot (A + B)$ Property 3
 $= 1 \cdot (A + B)$ Property 5
 $= A + B$ Property 4

5 $\bar{\bar{A}} = A$

prove let $\bar{\bar{A}} = X$

thus $X \cdot \bar{A} = 0$ $X + \bar{A} = 1$ Property 5

but $A \cdot \bar{A} = 0$ $A + \bar{A} = 1$ Property 5

Since both X and A obey Property 5, According to the uniqueness, we have : $A = X$

由于 $\bar{\bar{A}} = X$, 所以 $\bar{\bar{A}} = A$





6 $\overline{A+B} = \overline{A} \cdot \overline{B}$ $\overline{A \cdot B} = \overline{A} + \overline{B}$

$$\begin{aligned}(\overline{A} \cdot \overline{B}) + (A + B) &= (\overline{A} \cdot \overline{B} + A) + B \\&= (\overline{B} + A) + B \\&= A + (B + \overline{B}) \\&= A + 1 \\&= 1\end{aligned}$$

$$\begin{aligned}(\overline{A} \cdot \overline{B}) \cdot (A + B) &= \overline{A} \cdot \overline{B} \cdot A + \overline{A} \cdot \overline{B} \cdot B \\&= 0 + 0 \\&= 0\end{aligned}$$

$$\overline{A+B} = \overline{A} \cdot \overline{B}$$





DeMorgan's theorem (very important!)

$$\overline{X+Y} = \overline{X} \cdot \overline{Y}$$

break (or connect) the bar & change the sign

$$\overline{X \cdot Y} = \overline{X} + \overline{Y}$$

break (or connect) the bar & change the sign





Theorem 7 $A \cdot B + A \cdot \bar{B} = A$

$$(A + B) \cdot (A + \bar{B}) = A$$





Theorem 8

8

$$A \cdot B + \bar{A} \cdot C + B \cdot C = A \cdot B + \bar{A} \cdot C$$

$$(A + B) \cdot (\bar{A} + C) \cdot (B + C) = (A + B) \cdot (\bar{A} + C)$$

$$A \cdot B + \bar{A} \cdot C + B \cdot C$$

$$= A \cdot B + \bar{A} \cdot C + B \cdot C \cdot (A + \bar{A})$$

$$= A \cdot B + \bar{A} \cdot C + B \cdot C \cdot A + B \cdot C \cdot \bar{A}$$

$$= A \cdot B + A \cdot B \cdot C + \bar{A} \cdot C + \bar{A} \cdot B \cdot C$$

$$= A \cdot B \cdot (1 + C) + \bar{A} \cdot C \cdot (1 + B)$$

$$= A \cdot B \cdot (C + 1) + \bar{A} \cdot C \cdot (B + 1)$$

$$= A \cdot B + \bar{A} \cdot C$$





2.2.2 Important rules

Three important rules: .

I. Substitution rule

A logic equation containing a variable A , if all A variables are replaced by a logic function F , the equation is still correct. This rule is called the substitution rule.

E.g., in the logic function: $A(B+C)=AB+AC$, **we use $(E+D)$ to replace C , the logic equation is correct, i.e.,**

$$A [B+(C+D)] = AB+A(C+D)$$





- Meaning of substitution:

The substitution rule can use any logic functions to replay variables in the properties and theorems. It is not necessary to prove the replacement.

E.g., if use a logic function $F = f(B_1 B_2 \cdots B_n)$ to replace A in $\bar{A} + A = 1$, then we have $\bar{f}(B_1 B_2 \cdots B_n) + f(B_1 B_2 \cdots B_n) = 1$

Notice: When using substitution rules, all the same variable in the equation must be replaced by the unified function; otherwise, the substituted equation will not be correct.





II. Complementing rule

In a function, F , if change “.” to “+”, “+” to “.”, “0” to “1”, “1” to “0”, variables to complement variables, complement variables to variables; and retain the operation sequence as the same with that in F ; the new function $\bar{F}$ is the complement function of F .

ie: “.” $\longleftrightarrow$ “+”, “0” $\longleftrightarrow$ “1”, variable $\longleftrightarrow$ complement variable

E.g., a function $F = \bar{A} \cdot B + C \cdot \bar{D}$, by using the complement rule, we have

$$\bar{F} = (A + \bar{B}) \cdot (\bar{C} + D)$$





Notice: Retain the operation sequence as the same as that in the original function.

E.g., A function, $F = \bar{A} + \bar{B} \cdot (C + D\bar{E})$, by using the complementation rule, the result is:

$$\bar{F} = A \cdot [B + \bar{C} \cdot (\bar{D} + E)]$$

Rather than $\bar{F} = A \cdot B + \bar{C} \cdot \bar{D} + E$ **× ! incorrect**

III. Dual rule

In a function, F , if change “.” to “+”, “+” to “.”, “0” to “1”, “1” to “0”, the new logic function is the dual function of F , It is written as F' e.g.,

$$F = A B + \bar{B} (C + 0) \quad ; \quad F' = (A + B)(\bar{B} + C \cdot 1)$$





Notice: Retain the operation sequence as the same as that in the original function.

Property 1: if two logic equations, F and G, are equivalent, the dual equations of F and G are also equalized. This is called the dual rule.

Based on the dual rule, if two equations are proved to be equivalent, then, their dual equations are equivalent.

E. g. , if $AB + \bar{A}C + BC = AB + \bar{A}C$, dual the both sides of the equals symbol,

$$(A+B) (\bar{A}+C) (B+C) = (A+B) (\bar{A}+C)$$

Obviously, using duality rules can reduce the proofs of theorems and formulas by half.





2.2.3 Compound logic

In practical applications, the common gate circuits such as NAND, NOR, AND-OR-INVERT, and XOR gates are widely used. The logic relationship between the output and input of these gate circuits can be described by a complex operation consisting of the three kinds of basic operations. Therefore, this kind of logic relation is usually called compound logic, and the corresponding logic gates are called a compound gate.

I. NAND logic

The NAND logic is formed by the combination of the AND logic and NOT logic. The available logic function is expressed as:

$$F = \overline{A \cdot B \cdot C \cdots}$$

Logic function: As long as one of the variables $A, B, C, \dots$ is 0, the function F is 1; only when the variables $A, B, C, \dots$ are all 1, the function F is 0.

The implementation circuits of NAND function are called NAND gates.



Using NAND gates can achieve the AND, OR, and NOT three basic operations:

AND: $F = \overline{\overline{A} \cdot \overline{B} \cdot 1} = \overline{\overline{A} \cdot \overline{B}} = A \cdot B$

OR: $F = \overline{\overline{A} \cdot 1 \cdot \overline{B} \cdot 1} = \overline{\overline{A} \cdot \overline{B}} = A + B$

NOT: $F = \overline{A \cdot 1} = \overline{A}$

NAND gates can compose circuits that implement various logic functions; therefore, NAND gates are often called universal gates.





II. NOR logic

NOR logic is formed by the combination of two or more logics and can be expressed as

$$F = \overline{A + B + C + \dots}$$

Logic function: As long as one of the variables $A, B, C, \dots$ is 1, the function F is 0; only when the variables $A, B, C, \dots$ are all 0, the function F is 1.

Similarly, using NOR gates can also achieve the AND, OR, and NOT three basic operations:

$$\text{AND: } F = \overline{\overline{A + 0} + \overline{B + 0}} = \overline{\overline{A} + \overline{B}} = A \cdot B$$

$$\text{OR: } F = \overline{\overline{A + B} + 0} = \overline{\overline{A + B}} = A + B$$

$$\text{NOT: } F = \overline{A + 0} = \overline{A}$$

NOR gates can compose circuits that implement various logic functions; therefore, NOR gates are also called universal gates.





III. AND-OR-INVERT

The AND logic is formed by combining 3 basic logics. The expression of the logic function is

$$F = \overline{AB + CD + \dots}$$

Logic function: only if every AND item is 0, F is 1; otherwise F is 0.

An circuit is called an AND-OR-INVERT gate.

Obviously, logic circuits implementing various functions can be composed only with AND-OR-INVERT gates. However, in practice, such doing will generally be very uneconomical. Therefore, AND-OR-INVERT gates are mainly used to implement AND-OR-INVERT functions only. Only when it is necessary, a logical function expression may be transformed into a form of AND-OR-INVERT.





IV. XOR and X-NOR

1. XOR

XOR logic is a two-variable logic relationship that can be expressed as a logical function

$$F = A \oplus B = \overline{A}B + A\overline{B}$$

Logic function: The variables **A** and **B** have the same value, **F** is 0; the variables **A** and **B** have different values, **F** is 1.

The logic gate that realizes XOR operation is called "exclusive OR gate".

Based the definition, we have:

$$A \oplus 0 = A \quad A \oplus 1 = \overline{A}$$

$$A \oplus A = 0 \quad A \oplus \overline{A} = 1$$

If a XOR function has many variables, the variables can be grouped using each a pair of variables; or just calculates in turn.





e. g. , $F = A \oplus B \oplus C \oplus D$

$$= (A \oplus B) \oplus (C \oplus D) \quad (\text{Grouped with two inputs})$$

$$= [(A \oplus B) \oplus C] \oplus D \quad (\text{in turn})$$

Notice: An XOR function produces a true output when the odd number of 1 value input is true; produce a false output when the even number of 1 value input is false.

2. X-NOR

X-NOR logic is also a two-variable logic relationship that can be expressed as a logical function

$$F = A \odot B = \bar{A} \cdot \bar{B} + AB$$

where, “ $\odot$ ” is the logic symbol of X-NOR.

Logic function: The variables A and B have the different value, F is 0; the variables A and B have same values, F is 1.

The logic gate that realizes X-NOR operation is called "exclusive OR gate".





XOR and X-NOR are logic inverse of each other and logic dual of each other. We have:

$$\begin{aligned}\overline{A \oplus B} &= \overline{A\overline{B} + \overline{A}B} = (\overline{A\overline{B}})(\overline{\overline{A}B}) \\ &= \overline{A\overline{B}} + \overline{\overline{A}B} = A \odot B \\ (A \oplus B)' &= (\overline{A\overline{B}} + \overline{\overline{A}B})' = (\overline{A\overline{B}})(\overline{\overline{A}B}) \\ &= \overline{A\overline{B}} + \overline{\overline{A}B} = A \odot B\end{aligned}$$

Notice: An X-NOR function produces a false output when the odd number of 1 value input is true; produce a true output when the even number of 1 value input is false.

In terms of X-NOR is the complement of XOR, in practice applications, X-NOR gate is implemented by combining NOT and XNOR.

